

Cours PHP

Chapitre 1 – Introduction à la Programmation Orientée Objet (POO) en PHP

1.1. Pourquoi la POO ?

La **programmation orientée objet (POO)** permet de structurer le code en regroupant les données (attributs) et les comportements (méthodes) dans des **objets**.

Elle favorise la réutilisabilité, la maintenance et la modularité du code.

1.2. Création d'une classe et d'un objet

```
<?php
```

```
class Voiture {  
    public $marque;  
    public $couleur;  
  
    public function demarrer() {  
        echo "La voiture démarre.";  
    }  
}
```

```
$voiture1 = new Voiture();  
$voiture1->marque = "Toyota";  
$voiture1->couleur = "Rouge";  
$voiture1->demarrer();  
?>
```

1.3. Constructeurs et destructeurs

```
class Utilisateur {  
    public $nom;  
  
    public function __construct($nom) {  
        $this->nom = $nom;  
    }  
  
    public function __destruct() {  
        echo "L'objet est détruit.";  
    }  
}
```

```
$user = new Utilisateur("Alice");  
echo $user->nom;
```

1.4. Encapsulation, visibilité et propriétés

- `public` : accessible partout
- `private` : accessible uniquement dans la classe
- `protected` : accessible dans la classe et ses enfants

```
class CompteBancaire {  
    private $solde = 0;  
  
    public function deposer($montant) {  
        $this->solde += $montant;  
    }  
  
    public function consulterSolde() {  
        return $this->solde;  
    }  
}
```

1.5. Héritage

```
class Animal {  
    public function parler() {  
        echo "Un bruit";  
    }  
}  
  
class Chien extends Animal {  
    public function parler() {  
        echo "Wouf !";  
    }  
}
```

```
$chien = new Chien();  
$chien->parler(); // Affiche "Wouf !"
```

1.6. Polymorphisme et classes abstraites

```
abstract class Forme {  
    abstract public function aire();  
}  
  
class Carre extends Forme {  
    private $côté;  
    public function __construct($c) { $this->côté = $c; }  
    public function aire() { return $this->côté * $this->côté; }  
}
```

```
$c = new Carre(5);  
echo $c->aire(); // 25
```

Exercice : Crée une hiérarchie de classes pour modéliser des **comptes bancaires** (courant, épargne).

Chapitre 2 – Manipulation de Bases de Données avec PDO

2.1. Connexion à la base de données

```
try {
    $pdo = new PDO("mysql:host=localhost;dbname=etudiants", "root", "");
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Connexion réussie";
} catch (PDOException $e) {
    echo "Erreur : " . $e->getMessage();
}
```

2.2. Lecture des données (SELECT)

```
$sql = "SELECT * FROM eleves";
$stmt = $pdo->query($sql);
while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
    echo $row['nom'] . "<br>";
}
```

2.3. Requêtes préparées (sécurité contre les injections SQL)

```
$sql = "SELECT * FROM eleves WHERE matricule = :matricule";
$stmt = $pdo->prepare($sql);
$stmt->execute(['matricule' => 'ET123']);
$result = $stmt->fetch(PDO::FETCH_ASSOC);
```

2.4. Insertion de données

```
$sql = "INSERT INTO eleves (nom, prenom) VALUES (:nom, :prenom)";
$stmt = $pdo->prepare($sql);
$stmt->execute(['nom' => 'Zohou', 'prenom' => 'Jérôme']);
```

2.5. Modification et suppression

```
// UPDATE
$stmt = $pdo->prepare("UPDATE eleves SET nom = :nom WHERE id = :id");
$stmt->execute(['nom' => 'Zana', 'id' => 1]);

// DELETE
$stmt = $pdo->prepare("DELETE FROM eleves WHERE id = :id");
```

```
$stmt->execute(['id' => 1]);
```

TP : Crée une application PHP qui gère une table `etudiants` (CRUD complet avec PDO).

Chapitre 3 – Sécurité en PHP

3.1. Éviter les injections SQL

Toujours utiliser des **requêtes préparées** avec PDO.

3.2. Protection contre les failles XSS

```
echo htmlspecialchars($_POST['nom']);
```

3.3. Sécurité des formulaires (token CSRF)

```
$_SESSION['token'] = bin2hex(random_bytes(32));
```

3.4. Gestion sécurisée des mots de passe

```
$hash = password_hash("monmotdepasse", PASSWORD_DEFAULT);  
if (password_verify("monmotdepasse", $hash)) {  
    echo "Connexion réussie";  
}
```